UNIT 4

Computer Instructions

Computer instructions are a set of machine language instructions that a particular processor understands and executes. A computer performs tasks on the basis of the instruction provided.

An instruction comprises of groups called fields. These fields include:

- The Operation code (Opcode) field which specifies the operation to be performed.
- The Address field which contains the location of the operand, i.e., register or memory location.
- The Mode field which specifies how the operand will be located.

A basic computer has three instruction code formats which are:

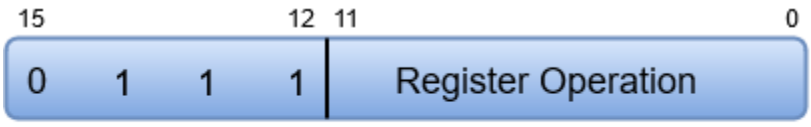
1. Memory - reference instruction
2. Register - reference instruction
3. Input-Output instruction

Memory - reference instruction

(Opcode = 000 through 110)

In Memory-reference instruction, 12 bits of memory is used to specify an address and one bit to specify the addressing mode 'I'.

Register - reference instruction



(Opcode = 111, I = 0)

The Register-reference instructions are represented by the Opcode 111 with a 0 in the leftmost bit (bit 15) of the instruction.

Note: The Operation code (Opcode) of an instruction refers to a group of bits that define arithmetic and logic operations such as add, subtract, multiply, shift, and compliment.

A Register-reference instruction specifies an operation on or a test of the AC (Accumulator) register.

Input-Output instruction

(Opcode = 111, I = 1)

Just like the Register-reference instruction, an Input-Output instruction does not need a reference to memory and is recognized by the operation code 111 with a 1 in the leftmost bit of the instruction. The remaining 12 bits are used to specify the type of the input-output operation or test performed.

Note

- The three operation code bits in positions 12 through 14 should be equal to 111. Otherwise, the instruction is a memory-reference type, and the bit in position 15 is taken as the addressing mode I.
- When the three operation code bits are equal to 111, control unit inspects the bit in position 15. If the bit is 0, the instruction is a register-reference type. Otherwise, the instruction is an input-output type having bit 1 at position 15.

Instruction Set Completeness

A set of instructions is said to be complete if the computer includes a sufficient number of instructions in each of the following categories:

- Arithmetic, logical and shift instructions
- A set of instructions for moving information to and from memory and processor registers.
- Instructions which controls the program together with instructions that check status conditions.
- Input and Output instructions

Arithmetic, logic and shift instructions provide computational capabilities for processing the type of data the user may wish to employ.

A huge amount of binary information is stored in the memory unit, but all computations are done in processor registers. Therefore, one must possess the capability of moving information between these two units.

Program control instructions such as branch instructions are used change the sequence in which the program is executed.

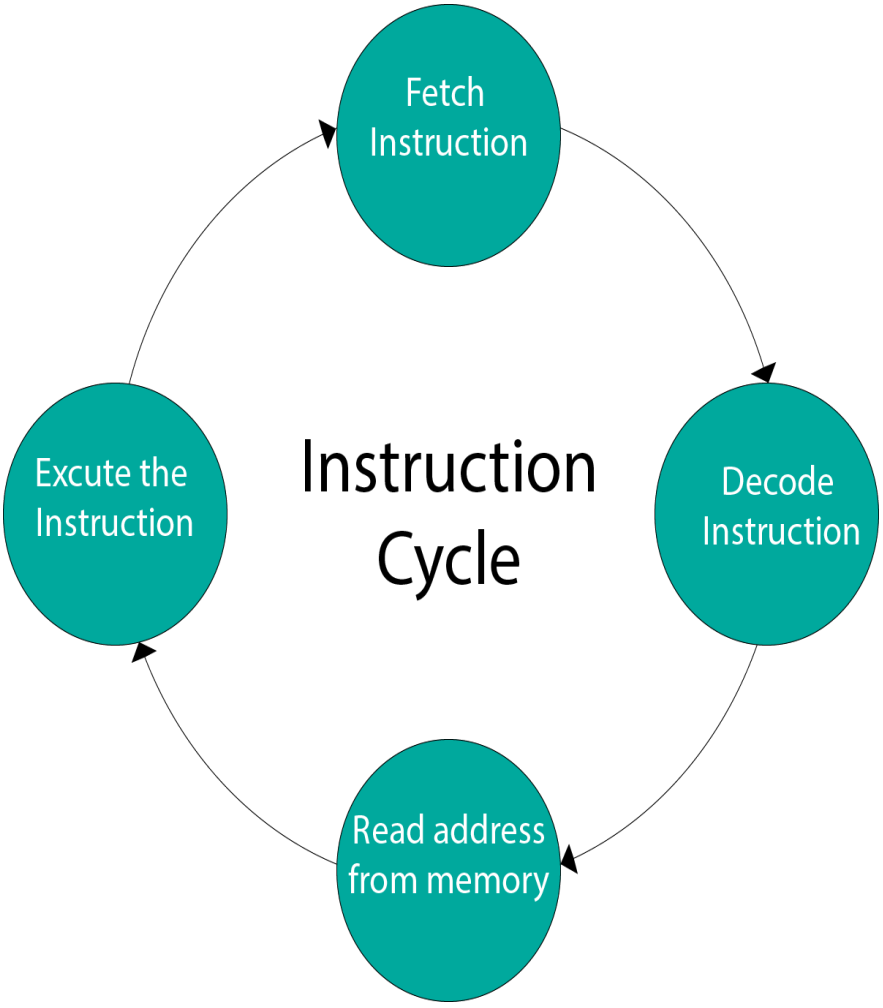
Input and Output instructions act as an interface between the computer and the user. Programs and data must be transferred into memory, and the results of computations must be transferred back to the user.

Instruction Cycle

A program residing in the memory unit of a computer consists of a sequence of instructions. These instructions are executed by the processor by going through a cycle for each instruction.

In a basic computer, each instruction cycle consists of the following phases:

1. Fetch instruction from memory.
2. Decode the instruction.
3. Read the effective address from memory.
4. Execute the instruction.



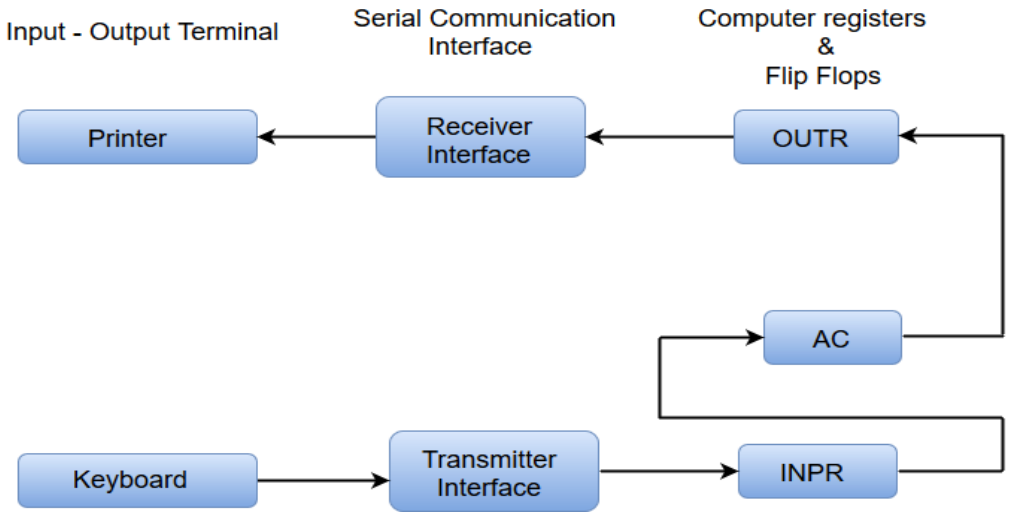
Input-Output Configuration

In computer architecture, input-output devices act as an interface between the machine and the user.

Instructions and data stored in the memory must come from some input device. The results are displayed to the user through some output device.

The following block diagram shows the input-output configuration for a basic computer.

Input - Output Configuration:



- The input-output terminals send and receive information.
- The amount of information transferred will always have eight bits of an alphanumeric code.
- The information generated through the keyboard is shifted into an input register 'INPR'.
- The information for the printer is stored in the output register 'OUTR'.
- Registers INPR and OUTR communicate with a communication interface serially and with the AC in parallel.
- The transmitter interface receives information from the keyboard and transmits it to INPR.
- The receiver interface receives information from OUTR and sends it to the printer serially.

Design of a Basic Computer

A basic computer consists of the following hardware components.

1. A memory unit with 4096 words of 16 bits each
2. Registers: AC (Accumulator), DR (Data register), AR (Address register), IR (Instruction register), PC (Program counter), TR (Temporary register), SC (Sequence Counter), INPR (Input register), and OUTR (Output register).
3. Flip-Flops: I, S, E, R, IEN, FGI and FGO

Note: FGI and FGO are corresponding input and output flags which are considered as control flip-flops.

1. Two decoders: a 3 x 8 operation decoder and 4 x 16 timing decoder
2. A 16-bit common bus
3. Control Logic Gates

4. The Logic and Adder circuits connected to the input of AC.

instruction set

An instruction set is a group of commands for a central processing unit (CPU) in machine language. The term can refer to all possible instructions for a CPU or a subset of instructions to enhance its performance in certain situations.

All CPUs have instruction sets that enable commands directing the CPU to switch the relevant transistors. The instructions tell the CPU to perform tasks. Some instructions are simple read, write and move commands that direct data to different hardware elements.

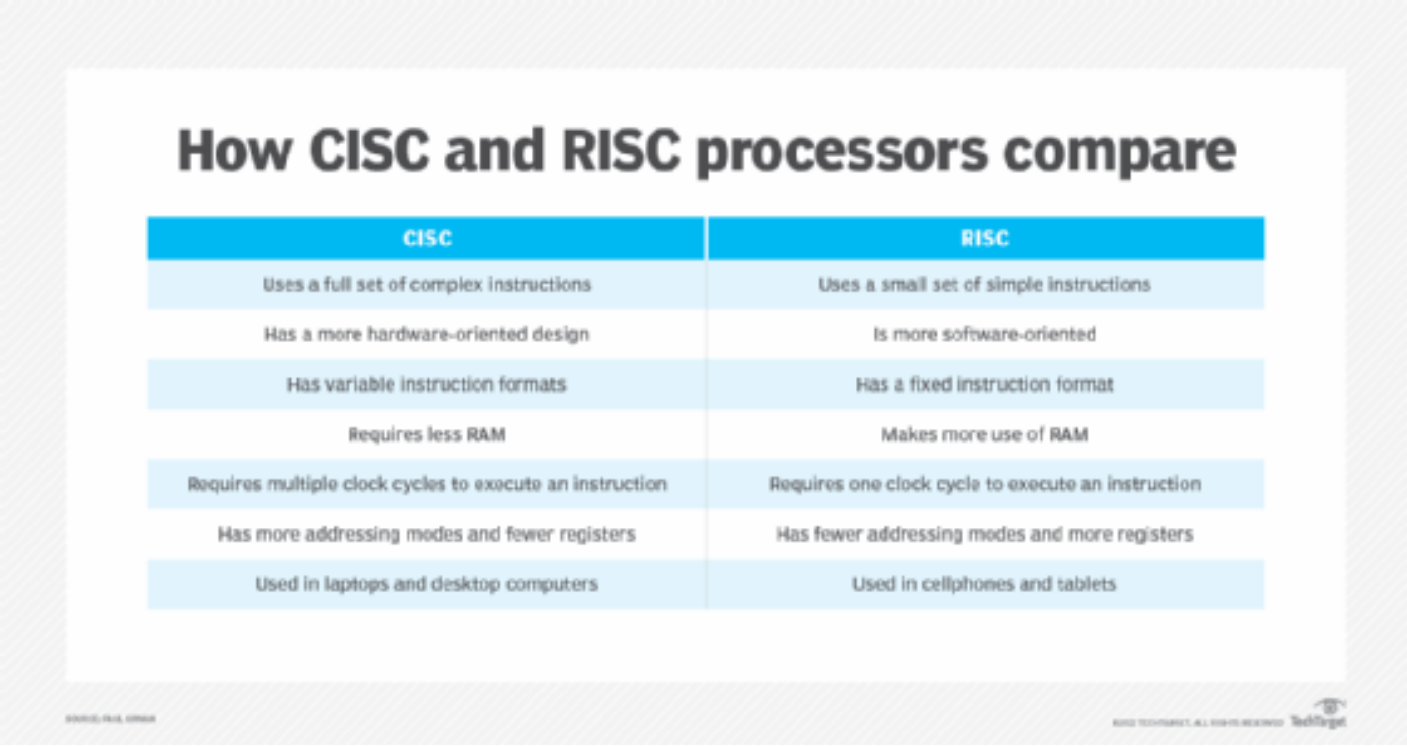
The instructions are made up of a specific number of bits. For instance, The CPU's instructions might be 8 bits, where the first 4 bits make up the operation code that tells the computer what to do. The next 4 bits are the operand, which tells the computer the data that should be used.

The length of an instruction set can vary from as few as 4 bits to many hundreds. Different instructions in some instruction set architectures (ISAs) have different lengths. Other ISAs have fixed-length instructions.

What are some types of instruction sets?

The various types of instruction sets include the following:

- **Complex instruction set computer.** CISC processors have an additional microcode or microprogramming layer where instructions act as small programs. Programmable instructions are stored in fast memory and can be updated. More instructions are included in CISC instruction sets than in other types of instruction sets. A single instruction can initiate multiple actions by the computer, such as a single add command launching multiple memory access load and store instructions.
- **Reduced instruction set computer.** RISC architecture has hard-wired control. It does not require microcode, but has a greater base instruction set. RISC also uses a smaller and more compact instruction set with a fixed instruction format. RISC processors are designed to process faster and more efficiently.
- **Enhancement instruction sets.** These instruction types are more familiar because they are often used in marketing CPUs. Examples of this go back to the 166-megahertz Intel Pentium with MultiMedia Extensions (MMX) technologies. It was introduced in 1996 and marketed with enhanced Intel CPU multimedia performance. MMX refers to the extended instruction set.



CISC processers are complex and versatile, while RISC processors are simpler and more efficient. See how they compare.

Why are instruction sets important

The ISA is fundamental for building fast, efficient computers that optimize memory and processing resources. It specifies the following supported capabilities:

- instructions
- data types
- processor registers
- main memory hardware
- input/output model
- addressing modes

Programmers and system engineers rely on the ISA for guidance on how to program various activities.

Instruction sets work with other important parts of a computer, such as compilers and interpreters. Those components translate high-level programming code into machine code that the processor can understand.

Think of the ISA as a programmer's gateway into the inner workings of a computer. It is essential when working with application development, assembly language and compilers. It sets the rules for the hardware and software interface, defines what the CPU does and ensures compatibility.

How are instruction set commands used?

The following are three main ways instruction set commands are used:

1. **Data handling and memory management.** Instruction set commands are used when setting a register to a specific value, copying data from memory to a register or vice versa, and reading and writing data.
2. **Arithmetic and logic operations and activities.** These commands include add, subtract, multiply, divide and compare, which examines values in two registers to see if one is greater or less than the other.
3. **Control flow activities.** One example is branch, which instructs the system to go to another location and execute commands there. Another is jump, which moves to a specific memory location or address.

Instruction Formats (Zero, One, Two and Three Address Instruction)

A computer performs a task based on the instruction provided. Instruction in computers comprises groups called fields. These fields contain different information as for computers everything is in 0 and 1 so each field has different significance based on which a CPU decides what to perform. The most common fields are:

- Operation field specifies the operation to be performed like addition.
- Address field which contains the location of the operand, i.e., register or memory location.
- Mode field which specifies how operand is to be founded.

Instruction is of variable length depending upon the number of addresses it contains. Generally, CPU organization is of three types based on the number of address fields:

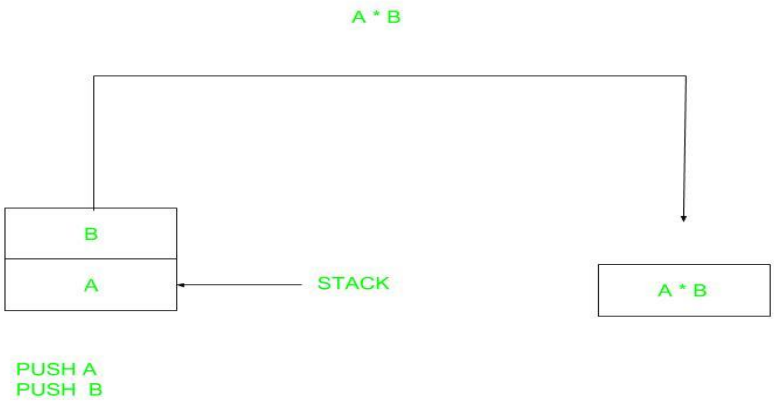
1. Single Accumulator organization
2. General register organization
3. Stack organization

In the first organization, the operation is done involving a special register called the accumulator. In second on multiple registers are used for the computation purpose. In the third organization the work on stack basis operation due to which it does not contain any address field. Only a single organization doesn't need to be applied, a blend of various organizations is mostly what we see generally.

Based on the number of address, instructions are classified as:

Note that we will use X = (A+B)*(C+D) expression to showcase the procedure.

1. **Zero Address Instructions –**



A stack-based computer does not use the address field in the instruction. To evaluate an expression first it is converted to reverse Polish Notation i.e. Postfix Notation.

Expression: X = (A+B)*(C+D)

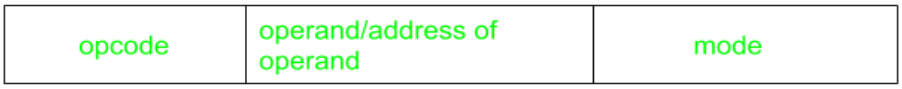
Postfixed : X = AB+CD+*

TOP means top of stack

M[X] is any memory location

PUSH	A	TOP = A
PUSH	B	TOP = B
ADD		TOP = A+B
PUSH	C	TOP = C
PUSH	D	TOP = D
ADD		TOP = C+D
MUL		TOP = (C+D)*(A+B)
POP	X	M[X] = TOP

2 .One Address Instructions –
This uses an implied ACCUMULATOR register for data manipulation. One operand is in the accumulator and the other is in the register or memory location. Implied means that the CPU already knows that one operand is in the accumulator so there is no need to specify it.



Expression: X = (A+B)*(C+D)

AC is accumulator

M[] is any memory location

M[T] is temporary location

LOAD A AC = M[A]

ADD B AC = AC + M[B]

STORE T M[T] = AC

LOAD C AC = M[C]

ADD D AC = AC + M[D]

MUL T AC = AC * M[T]

STORE X M[X] = AC

3.Two Address Instructions –
This is common in commercial computers. Here two addresses can be specified in the instruction. Unlike earlier in one address instruction, the result was stored in the accumulator, here the result can be stored at different locations rather than just accumulators, but require more number of bit to represent address.

Here destination address can also contain operand.

Expression: X = (A+B)*(C+D)

R1, R2 are registers

M[] is any memory location

MOV R1, A R1 = M[A]

ADD R1, B R1 = R1 + M[B]

MOV R2, C R2 = C

ADD R2, D R2 = R2 + D

MUL R1, R2 R1 = R1 * R2

MOV X, R1 M[X] = R1

4.Three Address Instructions –
This has three address field to specify a register or a memory location. Program created are much short in size but number of bits per instruction increase. These instructions make creation of program much easier but it does not mean that program will run much faster because now instruction only contain more information but each micro operation (changing content of register, loading address in address bus etc.) will be performed in one cycle only.

Expression: X = (A+B)*(C+D)

R1, R2 are registers

M[] is any memory location

ADD R1, A, B R1 = M[A] + M[B]

ADD R2, C, D R2 = M[C] + M[D]

MUL X, R1, R2 M[X] = R1 * R2

What are the types of Addressing Modes?
The operands of the instructions can be located either in the main memory or in the CPU registers. If the operand is placed in the main memory, then the instruction provides the location address in the operand field. Many methods are followed to specify the operand address. The different methods/modes for specifying the operand address in the instructions are known as addressing modes.

Types of Addressing Modes

There are various types of Addressing Modes which are as follows −

Implied Mode − In this mode, the operands are specified implicitly in the definition of the instruction. For example, the instruction "complement accumulator" is an implied-mode instruction because the operand in the accumulator register is implied in the definition of the instruction. All register reference instructions that use an accumulator are implied-mode instructions.

Instruction format with mode field

Opcode	Mode	Address

Immediate Mode − In this mode, the operand is specified in the instruction itself. In other words, an immediate-mode instruction has an operand field instead of an address field. The operand field includes the actual operand to be used in conjunction with the operation determined in the instruction. Immediate-mode instructions are beneficial for initializing registers to a constant value.

Register Mode − In this mode, the operands are in registers that reside within the CPU. The specific register is selected from a register field in the instruction. A k-bit field can determine any one of the 2^k registers.

Register Indirect Mode − In this mode, the instruction defines a register in the CPU whose contents provide the address of the operand in memory. In other words, the selected register includes the address of the operand rather than the operand itself.

A reference to the register is then equivalent to specifying a memory address. The advantage of a register indirect mode instruction is that the address field of the instruction uses fewer bits to select a register than would have been required to specify a memory address directly.

Autoincrement or Autodecrement Mode &minuend; This is similar to the register indirect mode except that the register is incremented or decremented after (or before) its value is used to access memory. When the address stored in the register defines a table of data in memory, it is necessary to increment or decrement the register after every access to the table. This can be obtained by using the increment or decrement instruction.

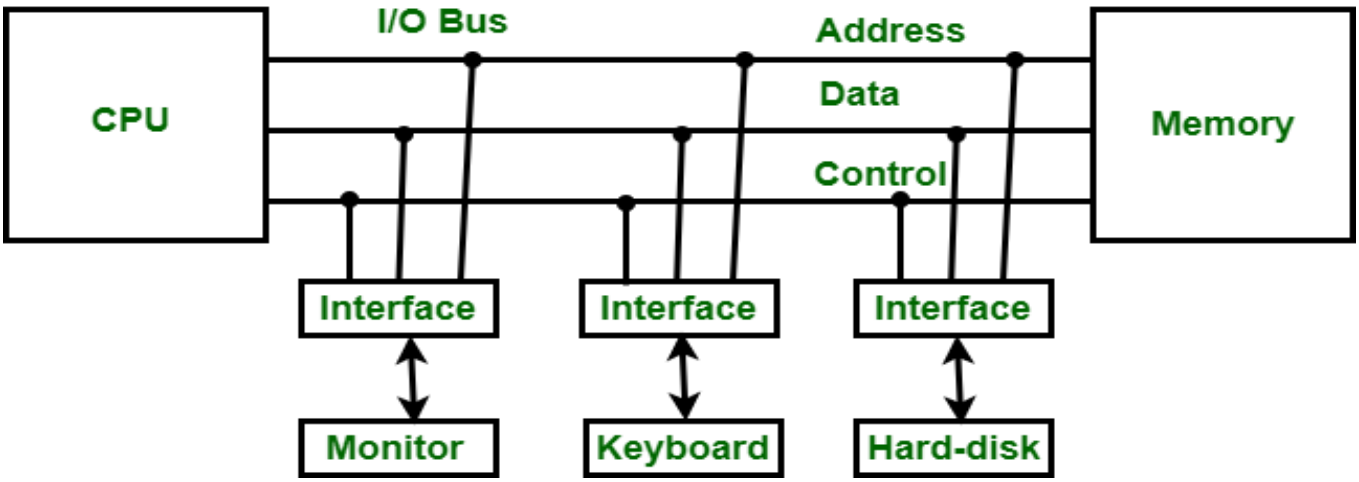
Direct Address Mode − In this mode, the effective address is equal to the address part of the instruction. The operand resides in memory and its address is given directly by the address field of the instruction. In a branch-type instruction, the address field specifies the actual branch address.

Indirect Address Mode − In this mode, the address field of the instruction gives the address where the effective address is stored in memory. Control fetches the instruction from memory and uses its address part to access memory again to read the effective address.

Indexed Addressing Mode − In this mode, the content of an index register is added to the address part of the instruction to obtain the effective address. The index register is a special CPU register that contains an index value. The address field of the instruction defines the beginning address of a data array in memory.

Introduction to Input-Output Interface

Input-Output Interface is used as an method which helps in transferring of information between the internal storage devices i.e. memory and the external peripheral device . A peripheral device is that which provide input and output for the computer, it is also called Input-Output devices. For Example: A keyboard and mouse provide Input to the computer are called input devices while a monitor and printer that provide output to the computer are called output devices. Just like the external hard-drives, there is also availability of some peripheral devices which are able to provide both input and output.



Input-Output Interface

In micro-computer base system, the only purpose of peripheral devices is just to provide **special communication links** for the interfacing them with the CPU. To resolve the differences between peripheral devices and CPU, there is a special need for communication links.
The major differences are as follows:

1. The nature of peripheral devices is electromagnetic and electro-mechanical. The nature of the CPU is electronic. There is a lot of difference in the mode of operation of both peripheral devices and CPU.
2. There is also a synchronization mechanism because the data transfer rate of peripheral devices are slow than CPU.
3. In peripheral devices, data code and formats are differ from the format in the CPU and memory.
4. The operating mode of peripheral devices are different and each may be controlled so as not to disturb the operation of other peripheral devices connected to CPU.

There is a special need of the additional hardware to resolve the differences between CPU and peripheral devices to supervise and synchronize all input and output devices.

Functions of Input-Output Interface:

1. It is used to synchronize the operating speed of CPU with respect to input-output devices.
2. It selects the input-output device which is appropriate for the interpretation of the input-output device.
3. It is capable of providing signals like control and timing signals.
4. In this data buffering can be possible through data bus.
5. There are various error detectors.
6. It converts serial data into parallel data and vice-versa.
7. It also convert digital data into analog signal and vice-versa.

Interrupts in 8085 microprocessor

When microprocessor receives any interrupt signal from peripheral(s) which are requesting its services, it stops its current execution and program control is transferred to a sub-routine by generating **CALL** signal and after executing sub-routine by generating **RET** signal again program control is transferred to main program from where it had stopped.
When microprocessor receives interrupt signals, it sends an acknowledgement (INTA) to the peripheral which is requesting for its service.

Interrupts can be classified into various categories based on different parameters:

1. **Hardware and Software Interrupts –**
   When microprocessors receive interrupt signals through pins (hardware) of microprocessor, they are known as *Hardware Interrupts*. There are 5 Hardware Interrupts in 8085 microprocessor. They are – *INTR, RST 7.5, RST 6.5, RST 5.5, TRAP*
   Software Interrupts are those which are inserted in between the program which means these are mnemonics of microprocessor. There are 8 software interrupts in 8085 microprocessor. They are – *RST 0, RST 1, RST 2, RST 3, RST 4, RST 5, RST 6, RST 7.*
2. **Vectored and Non-Vectored Interrupts –**
   Vectored Interrupts are those which have fixed vector address (starting address of sub-routine) and after executing these, program control is transferred to that address.
   Vector Addresses are calculated by the formula 8 * TYPE

INTERRUPT	VECTOR ADDRESS
TRAP (RST 4.5)	24 H
RST 5.5	2C H
RST 6.5	34 H
RST 7.5	3C H

3. For Software interrupts vector addresses are given by:

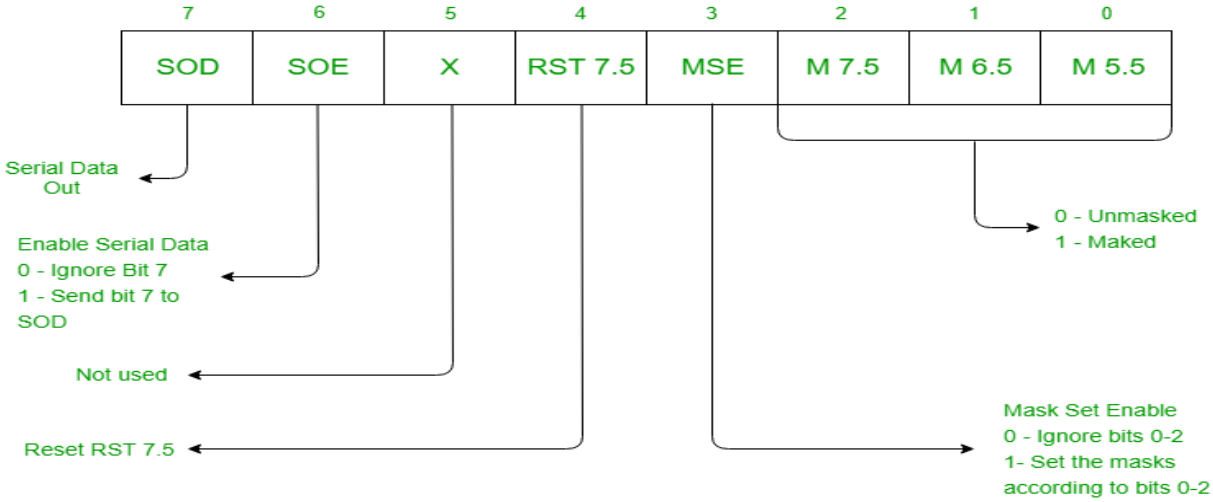
INTERRUPT	VECTOR ADDRESS
RST 0	00 H
RST 1	08 H
RST 2	10 H
RST 3	18 H
RST 4	20 H
RST 5	28 H
RST 6	30 H
RST 7	38 H

4. *Non-Vectored Interrupts* are those in which vector address is not predefined. The interrupting device gives the address of sub-routine for these interrupts. *INTR* is the only non-vectored interrupt in 8085 microprocessor.

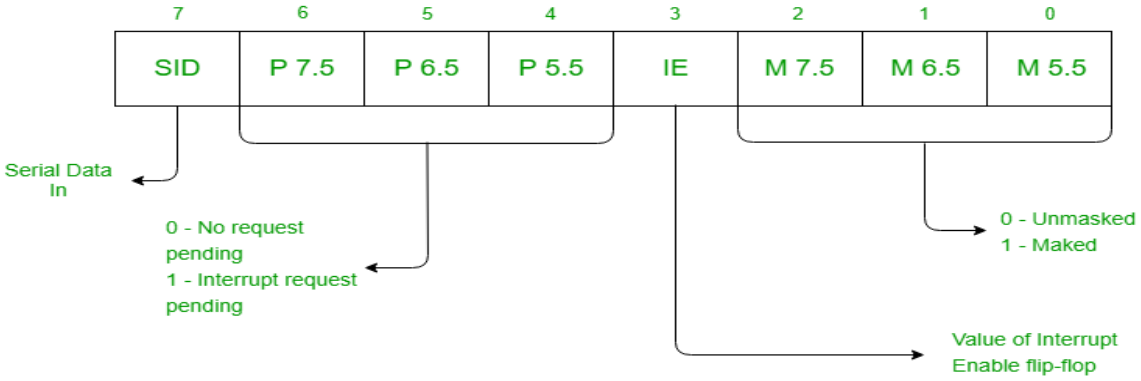
5. **Maskable and Non-Maskable Interrupts –**
   Maskable Interrupts are those which can be disabled or ignored by the microprocessor. These interrupts are either edge-triggered or level-triggered, so they can be disabled. *INTR, RST 7.5, RST 6.5, RST 5.5* are maskable interrupts in 8085 microprocessor.
   Non-Maskable Interrupts are those which cannot be disabled or ignored by microprocessor. *TRAP* is a non-maskable interrupt. It consists of both level as well as edge triggering and is used in critical power failure conditions.

Priority of Interrupts –
When microprocessor receives multiple interrupt requests simultaneously, it will execute the interrupt service request (ISR) according to the priority of the interrupts.

Instruction for Interrupts –
1. **Enable Interrupt (EI) –** The interrupt enable flip-flop is set and all interrupts are enabled following the execution of next instruction followed by EI. No flags are affected. After a system reset, the interrupt enable flip-flop is reset, thus disabling the interrupts. This instruction is necessary to enable the interrupts again (except TRAP).
2. **Disable Interrupt (DI) –** This instruction is used to reset the value of enable flip-flop hence disabling all the interrupts. No flags are affected by this instruction.
3. **Set Interrupt Mask (SIM) –** It is used to implement the hardware interrupts (RST 7.5, RST 6.5, RST 5.5) by setting various bits to form masks or generate output data via the Serial Output Data (SOD) line. First the required value is loaded in accumulator then SIM will take the bit pattern from it.



4. **Read Interrupt Mask (RIM) –** This instruction is used to read the status of the hardware interrupts (RST 7.5, RST 6.5, RST 5.5) by loading into the A register a byte which defines the condition of the mask bits for the interrupts. It also reads the condition of SID (Serial Input Data) bit on the microprocessor.

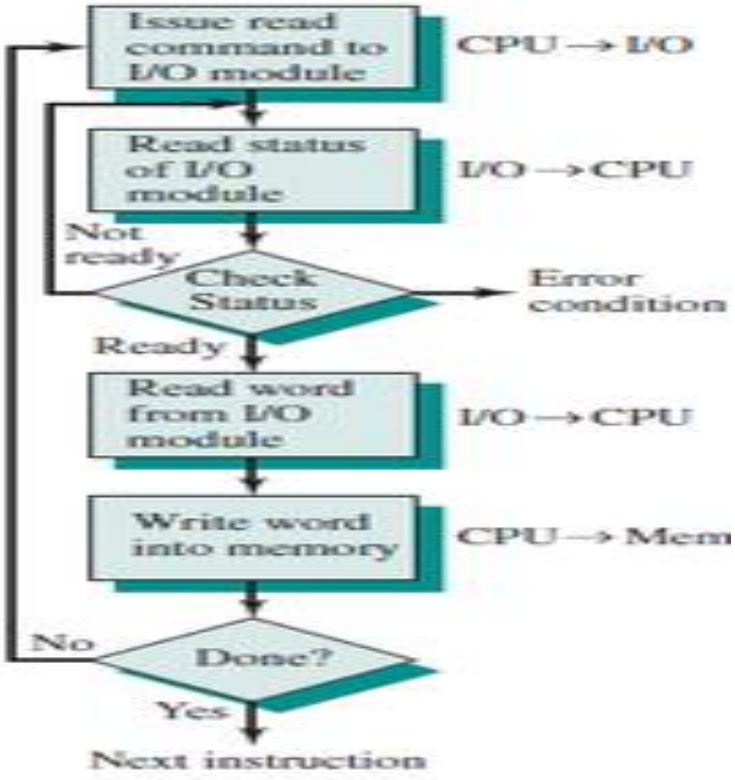


Programmed I/O

Programmable I/O is one of the I/O technique other than the interrupt-driven I/O and direct memory access (DMA). The programmed I/O was the most simple type of I/O technique for the exchanges of data or any types of communication between the processor and the external devices. With programmed I/O, data are exchanged between the processor and the I/O module. The processor executes a program that gives it direct control of the I/O operation, including sensing device status, sending a read or write command, and transferring the data. When the processor issues a command to the I/O module, it must wait until the I/O operation is complete. If the processor is faster than the I/O module, this is wasteful of processor time. The overall operation of the programmed I/O can be summaries as follow:

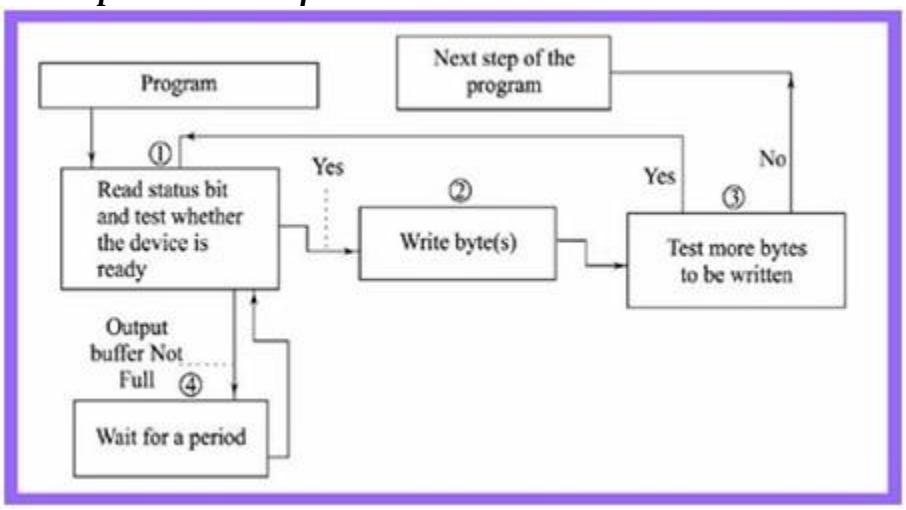
1. *The processor is executing a program and encounters an instruction relating to I/O operation.*
2. *The processor then executes that instruction by issuing a command to the appropriate I/O module.*
3. *The I/O module will perform the requested action based on the I/O command issued by the processor (READ/WRITE) and set the appropriate bits in the I/O status register.*
4. *The processor will periodically check the status of the I/O module until it find that the operation is complete.*

Programmed I/O Mode Input Data Transfer



1. Each input is read after first testing whether the device is ready with the input (a state reflected by a bit in a status register).
2. The program waits for the ready status by repeatedly testing the status bit and till all targeted bytes are read from the input device.
3. The program is in busy (non-waiting) state only after the device gets ready else in wait state.

Programmed I/O Mode Output Data Transfer



1. Each output written after first testing whether the device is ready to accept the byte at its output register or output buffer is empty.
2. The program waits for the ready status by repeatedly testing the status bit(s) and till all the targeted bytes are written to the device.
3. The program in busy (non-waiting) state only after the device gets ready else wait state.

I/O Commands

To execute an I/O-related instruction, the processor issues an address, specifying the particular I/O module and external device, and an I/O command. There are four types of I/O commands that an I/O module may receive when it is addressed by a processor:

- **Control**: Used to activate a peripheral and tell it what to do. For example, a magnetic-tape unit may be instructed to rewind or to move forward one record. These commands are tailored to the particular type of peripheral device.
- **Test**: Used to test various status conditions associated with an I/O module and its peripherals. The processor will want to know that the peripheral of interest is powered on and available for use. It will also want to know if the most recent I/O operation is completed and if any errors occurred.
- **Read**: Causes the I/O module to obtain an item of data from the peripheral and place it in an internal buffer. The processor can then obtain the data item by requesting that the I/O module place it on the data bus.
- **Write**: Causes the I/O module to take an item of data (byte or word) from the data bus and subsequently transmit that data item to the peripheral.

Direct Access Media (DMA) Controller in Computer Architecture

Direct Memory Access (DMA) :

DMA Controller is a hardware device that allows I/O devices to directly access memory with less participation of the processor. DMA controller needs the same old circuits of an interface to communicate with the CPU and Input/Output devices.

Fig-1 below shows the block diagram of the DMA controller. The unit communicates with the CPU through data bus and control lines. Through the use of the address bus and allowing the DMA and RS register to select inputs, the register within the DMA is chosen by the CPU. RD and WR are two-way inputs. When BG (bus grant) input is 0, the CPU can communicate with DMA registers. When BG (bus grant) input is 1, the CPU has relinquished the buses and DMA can communicate directly with the memory.

DMA controller registers :

The DMA controller has three registers as follows.

- **Address register –** It contains the address to specify the desired location in memory.
- **Word count register –** It contains the number of words to be transferred.
- **Control register –** It specifies the transfer mode.

Note –

All registers in the DMA appear to the CPU as I/O interface registers. Therefore, the CPU can both read and write into the DMA registers under program control via the data bus.

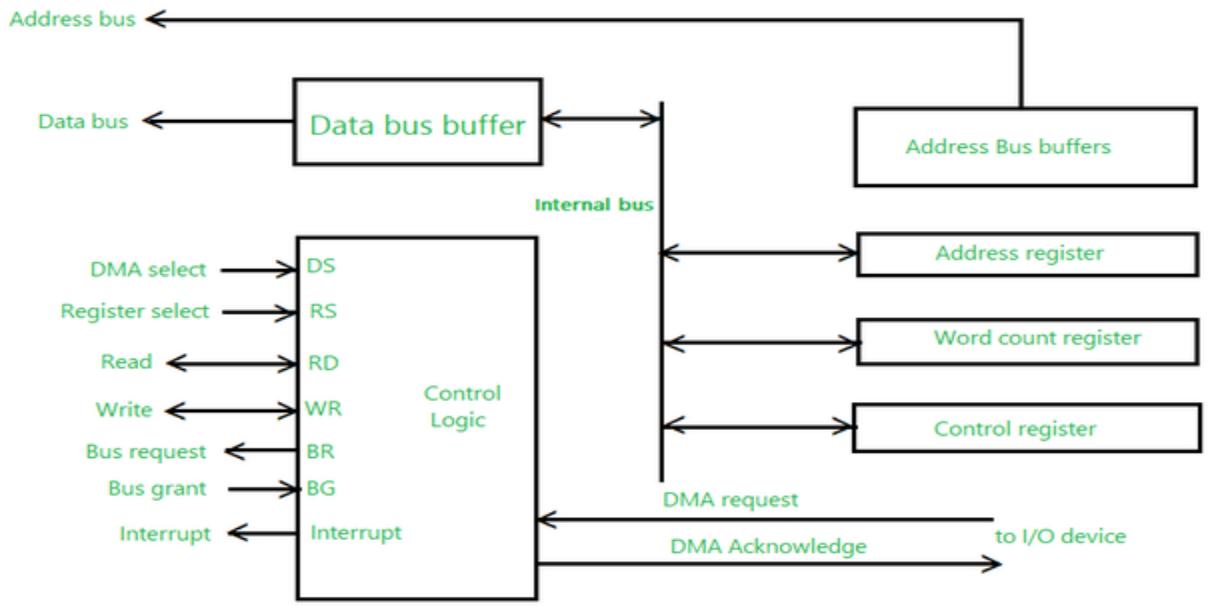


Fig 1- Block Diagram

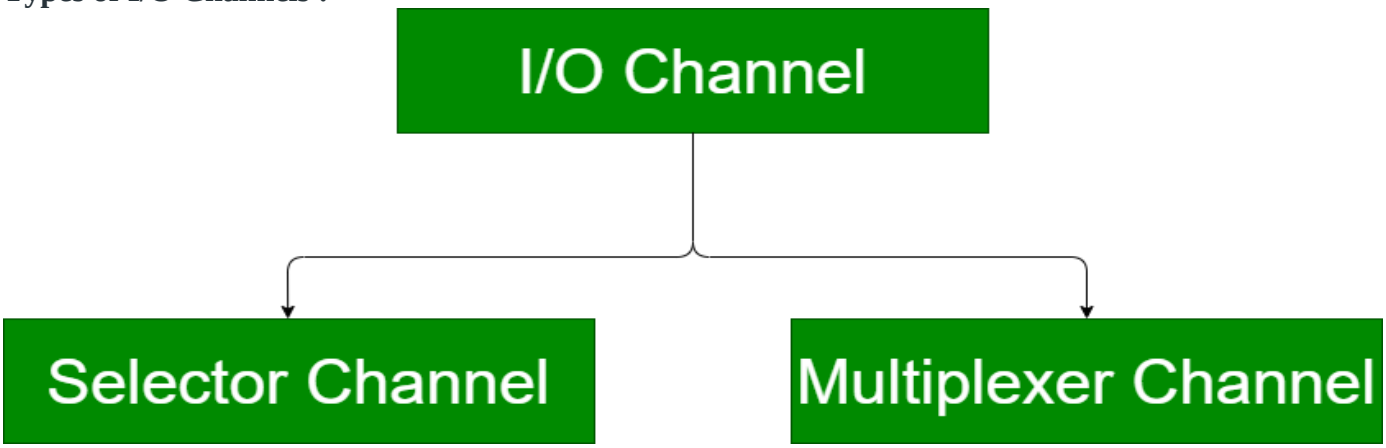
Explanation :
The CPU initializes the DMA by sending the given information through the data bus.

- The starting address of the memory block where the data is available (to read) or where data are to be stored (to write).
- It also sends word count which is the number of words in the memory block to be read or write.
- Control to define the mode of transfer such as read or write.
- A control to begin the DMA transfer.

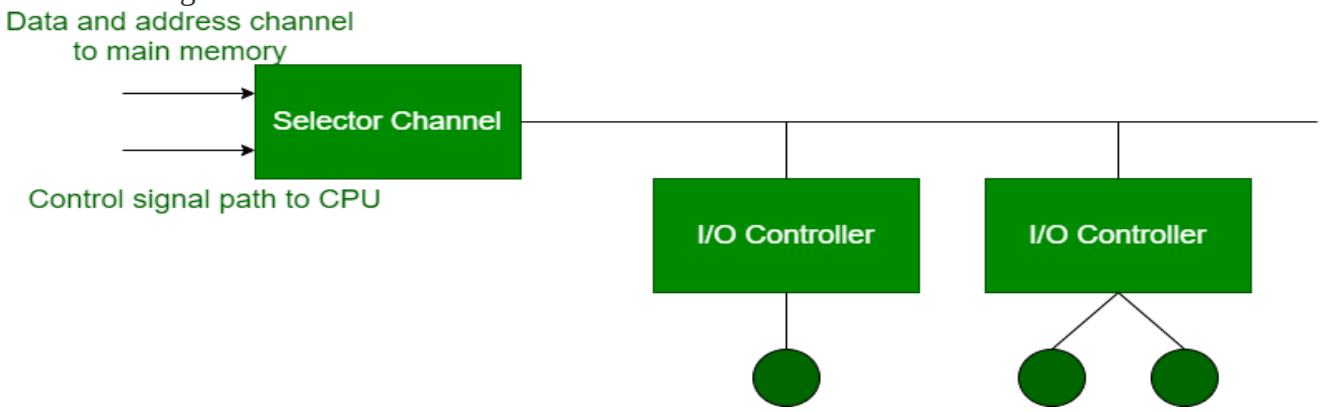
I/O Channels and its types

- Last Updated : 22 Apr, 2020

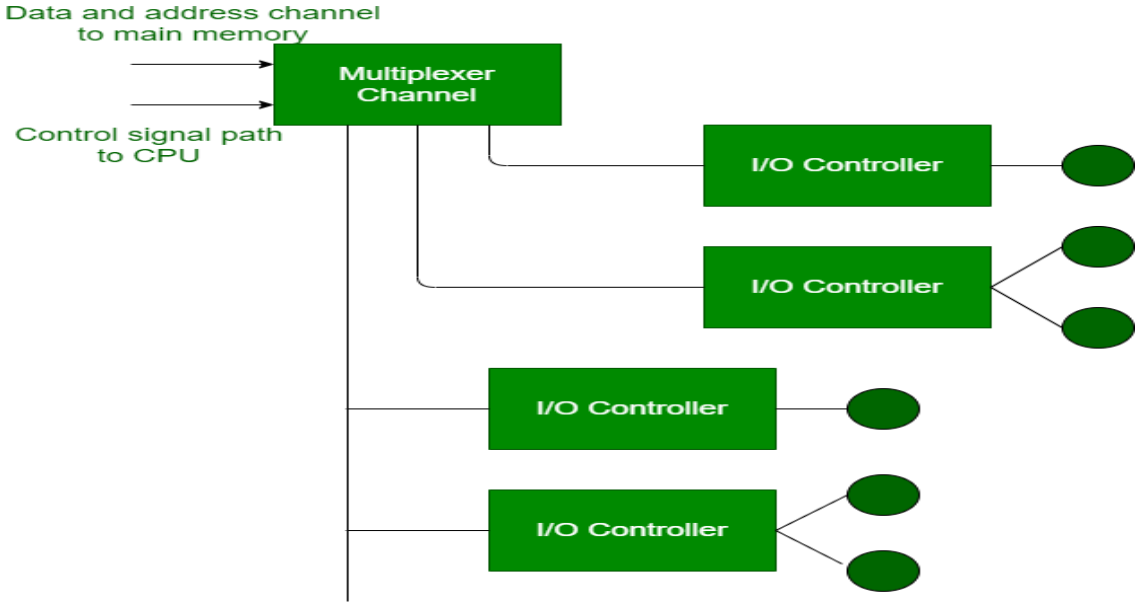
I/O Channel is an extension of the DMA concept. It has ability to execute I/O instructions using special-purpose processor on I/O channel and complete control over I/O operations. Processor does not execute I/O instructions itself. Processor initiates I/O transfer by instructing the I/O channel to execute a program in memory.
Program specifies – Device or devices, Area or areas of memory, Priority, and Error condition actions
Types of I/O Channels :



1. Selector Channel :
Selector channel controls multiple high-speed devices. It is dedicated to the transfer of data with one of the devices. In selector channel, each device is handled by a controller or I/O module. It controls the I/O controllers shown in the figure.



2. Multiplexer Channel :
Multiplexer channel is a DMA controller that can handle multiple devices at the same time. It can do block transfers for several devices at once.



Two types of multiplexers are used in this channel:

1. **Byte Multiplexer –**
   It is used for low-speed devices. It transmits or accepts characters. Interleaves bytes from several devices.
2. **Block Multiplexer –**
   It accepts or transmits block of characters. Interleaves blocks of bytes from several devices. Used for high-speed devices.

1. **Interrupt- initiated I/O:** Since in the above case we saw the CPU is kept busy unnecessarily. This situation can very well be avoided by using an interrupt driven method for data transfer. By using interrupt facility and special commands to inform the interface to issue an interrupt request signal whenever data is available from any device. In the meantime the CPU can proceed for any other program

execution. The interface meanwhile keeps monitoring the device. Whenever it is determined that the device is ready for data transfer it initiates an interrupt request signal to the computer. Upon detection of an external interrupt signal the CPU stops momentarily the task that it was already performing, branches to the service program to process the I/O transfer, and then return to the task it was originally performing.

Note: Both the methods programmed I/O and Interrupt-driven I/O require the active intervention of the processor to transfer data between memory and the I/O module, and any data transfer must transverse a path through the processor. Thus both these forms of I/O suffer from two inherent drawbacks.

- The I/O transfer rate is limited by the speed with which the processor can test and service a device.
- The processor is tied up in managing an I/O transfer; a number of instructions must be executed for each I/O transfer.

The Input-Process-Output Concept

A computer is an electronic device that accepts data, processes data, generates output, and stores data. The concept of generating output information from the input 4 data is also referred to as input-process-output concept.

The input-process-output concept of the computer is explained as follows—

Input

The computer accepts input data from the user via an input device like keyboard. The input data can be characters, word, text, sound, images, document, etc.

Process

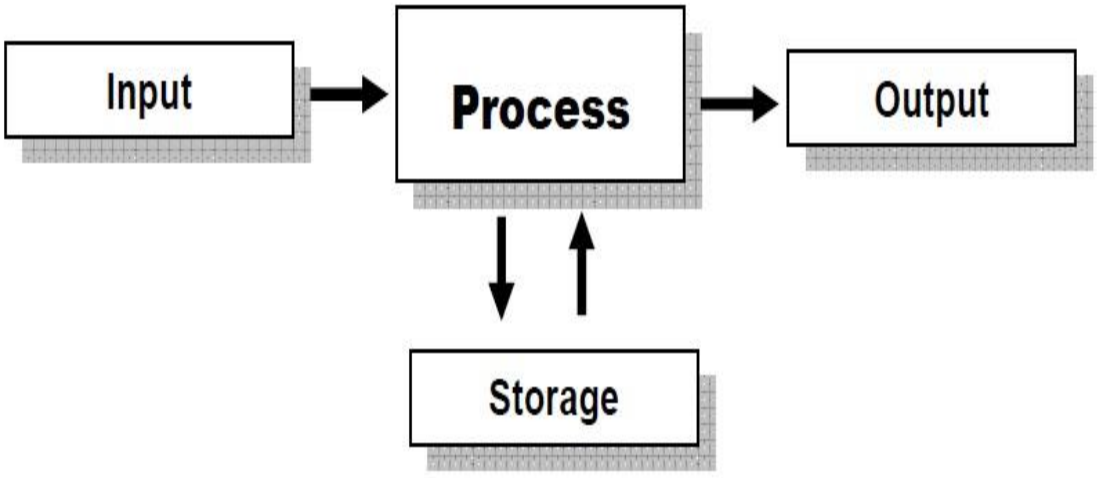
The computer processes the input data. For this, it performs some actions on the data by using the instructions or program given by the user of the data. The action could be an arithmetic or logic calculation, editing, modifying a document, etc. During processing, the data, instructions and the output are stored temporarily in the computer's main memory.

Output

The output is the result generated after the processing of data. The output may be in the form of text, sound, image, document, etc. The computer may display the output on a monitor, send output to the printer for printing, play the output, etc.

Storage

The input data, instructions and output are stored permanently in the secondary storage devices like disk or tape. The stored data can be retrieved later, whenever needed.



The Input-Process-Output Concept